Redis & Celery revision notes

01 · Redis from first principles

For Rahul · Revised 18 October 2026

Detailed foundations first; application patterns later. Highlighted questions come from the Redis/Celery discussion. Commands use temporary `learn:` keys so the examples are easy to distinguish from project data.

What Redis actually stores

Redis is a separate server that maintains a collection of **keys and values**. A key is a name you choose. Its value has a Redis data type: string, hash, list, set or sorted set, among others. The type determines the operations available on that value.

```
Key                          Value                        Type
learn:product:101:name        "Mouse"                      string
learn:product:101             {name: Mouse, price: 1500}    hash
learn:recent:user:7           ["101", "203", "108"]        list
```

Each key has one current value in the selected Redis database. Colons make names readable; Redis does not interpret them as folders or relationships. Different keys can have different types. GET reads a string; use HGET for a hash field and LRANGE for list elements.

Server, client and command

Redis server: the running process that owns the data and executes commands. **redis-cli:** a terminal client. **redis-py:** a Python library that sends Redis commands. **Django:** your application, which uses a client to communicate with Redis. Installing `pip install redis` installs the Python client, not the server.

```
Browser → Django API → PostgreSQL
                    → redis-py / Django cache → Redis server
redis-cli ──────────────────────────────────────→ Redis server
```

Why it can help

Redis keeps its working dataset primarily in RAM. More importantly for caching, it can reuse a prepared result: Django avoids repeating database queries and serialization. PostgreSQL also caches data in RAM, so “SQL always reads disk” is an incorrect explanation.

A cache is a reusable copy of data. In your project, PostgreSQL remains the source of truth for products, prices, orders and inventory. A lost product cache can be rebuilt; losing the only record of an order would be a different problem. Redis also supports counters, rankings and coordination, beyond caching.

Lifetime and durability

TTL limits how long a key may live. **Eviction** removes keys under memory pressure, potentially before TTL. Redis can persist to disk: **RDB** saves snapshots; **AOF** records writes. Recovery and possible data loss depend on configuration. Different Redis database numbers share the same server resources and failures.

First-principles rule: choose Redis data by the operations you need, define how long it lives, and decide what the application should do if it disappears. Redis data types.

02 · Basic commands and expiration

Start the server and open a client

```
sudo apt install redis-server
sudo service redis-server start
redis-cli
```

Run these inside redis-cli. PONG confirms the server responds, not that Django is configured.

Command	Meaning / result
`PING`	Check server response: PONG
`SET learn:name "Mouse"`	Create/replace a string value; returns OK
`GET learn:name`	Read value; missing key returns nil
`EXISTS learn:name`	For this one key, 1 means present, 0 absent
`TYPE learn:name`	Inspect the Redis type; absent key gives none
`DEL learn:name`	Delete; returns number of keys removed
`INCR learn:views`	Add 1 atomically; missing counter becomes 1
`INCRBY learn:views 5`	Add 5 to the integer representation

Strings also hold counters and serialized data

A string holds bytes: text, JSON or an integer representation. INCR parses an integer and fails for text such as "Mouse". It is atomic, so concurrent increments do not lose updates.

Attach a lifetime to the key

```
SET learn:offer "sale" EX 60
TTL learn:offer
EXPIRE learn:offer 120
PTTL learn:offer
PERSIST learn:offer
SET learn:once "token" NX EX 10
```

EX 60: expire after 60 seconds. **EXPIRE:** set/replace an existing key's lifetime. **TTL:** remaining whole seconds. **PTTL:** remaining milliseconds. **PERSIST:** remove the expiry so the key no longer has a TTL.

TTL result	Meaning
0 or positive	Key exists and is expiring; 0 means less than a second remains
-1	Key exists with no expiry
-2	Key does not exist

NX sets only if absent; **XX** sets only if present. `SET ... NX EX ...` combines the condition and expiry in one command. Plain SET overwrites the value and normally clears an old TTL; use EX again, or KEEPTTL when preserving it is intentional. INCR and HSET normally preserve an existing key TTL.

Remember: setting expiry once does not automatically refresh it on every GET. Your code must explicitly refresh it when a sliding lifetime is intended. Use `TTL`, not `til`. TTL reference.

03 · Key naming and SCAN

Design names around identity and purpose

```
ecommerce:dev:product:101
ecommerce:dev:recently-viewed:user:7
ecommerce:dev:most-viewed:2026-10-18
```

The environment prevents development/test collisions; the feature identifies the purpose; the final part identifies the object, user or time period. Include query options when they change the representation. Equivalent product UUID spellings should be normalized before forming keys, so one database product does not create several Redis entries.

Why SCAN exists

You normally know the exact key needed by a request. SCAN is for discovering keys incrementally during inspection or maintenance. `KEYS *` processes the whole matching keyspace in one command and can block a large live server. SCAN divides traversal across calls, letting other commands run between them.

```
SCAN 0 MATCH learn:product:* COUNT 2
```

```
Illustrative reply:
1) "6"
2) 1) "learn:product:101"
   2) "learn:product:203"
```

YOUR QUESTION — “Does it return just the cursor or keys as well?” Both. The first item is the next cursor; the second is this call's list of key names. Values are not included. Use the type-appropriate command to read a value.

Continue with the returned cursor:

```
SCAN 6 MATCH learn:product:* COUNT 2
```

The cursor is an opaque continuation token. It is not a page number, offset, number of keys processed or number remaining. Your real cursor can differ from 6 even with similar keys.

YOUR COUNTERQUESTION — “Can continuing from 6 return the first two keys again?” Yes. SCAN can return duplicates, so it is not distinct pagination. Collect results in a set when uniqueness matters, or make the processing safe to repeat.

Traversal rules

1. Begin with cursor 0.
2. Process every returned batch, including the last one.
3. Feed the returned cursor into the next call.
4. Stop only when the returned cursor is 0.

COUNT is a work hint, not an exact batch size. An empty batch with a nonzero cursor means continue. Order is unspecified. Traversal is not a snapshot: concurrent additions/removals may or may not appear. SCAN is incremental, but still consumes work.

In redis-py, `r.scan_iter(match='learn:*', count=100)` handles cursor iteration for you; it does not remove SCAN's duplicate or snapshot limitations. SCAN reference.

04 · Hashes: named fields in one key

The mental model

A hash stores field–value pairs under one Redis key, similar to a flat dictionary. The outer Redis key identifies the object; fields identify its properties. Values are byte strings, not automatically typed Python integers or nested dictionaries.

```
HSET learn:product:101 name "Mouse" priceCents 1500 stock 8
HGET learn:product:101 name
HMGET learn:product:101 name stock
HGETALL learn:product:101
HINCRBY learn:product:101 stock -1
```

HSET creates the hash if needed and adds/replaces fields. Updating priceCents leaves name and stock alone. HINCRBY performs an atomic integer increment on one field. In these examples EXPIRE would apply to the whole hash key.

Command	What it returns / does
`HGET key field`	One value; nil when that field is missing
`HMGET key f1 f2`	Values in requested order; missing positions are nil
`HGETALL key`	All fields and their values
`HEXISTS key field`	Whether that field exists
`HDEL key field`	Remove a field, leaving other fields
`HLEN key`	Number of fields
`HKEYS key` / `HVALS key`	All field names / all values

YOUR QUESTION — “What is HGETALL?” Every field–value pair in ONE hash, not every key in Redis. Interactive CLI output shows alternating fields and values; field order is not guaranteed. A missing hash gives an empty collection. Reading everything costs work proportional to the number of fields.

Your redis-py syntax question

```
r.hset("learn:product:101", mapping={
    "name": "Mouse", "priceCents": 1500, "stock": 8,
})
r.hget("learn:product:101", "name")
r.hmget("learn:product:101", ["name", "stock"])
r.hgetall("learn:product:101")
r.hincrby("learn:product:101", "stock", -1)
```

With `decode_responses=True`, HGETALL returns a dictionary such as `{'name': 'Mouse', 'priceCents': '1500', 'stock': '7'}`. Numeric-looking values are still strings; convert where needed. With default decoding, fields and values are bytes.

Hash or JSON string?

Use a hash when you need field reads, partial updates or atomic field counters. Use a serialized string when you normally read/replace the whole prepared result. JSON text stored by SET is still one Redis string; GET does not expose individual JSON fields as Redis hash fields.

Your token bucket keeps `tokens` and `last_updated` in a hash. Lua coordinates the complete read/refill/write sequence; using a hash alone does not make those steps atomic.

05 · Lists and sets

Lists: order matters, duplicates are allowed

A list is a sequence with left and right ends. Push adds elements; pop removes and returns one. Redis list indices start at zero; negative indices count from the end. Range end indices are inclusive.

```
RPUSH learn:recent "A" "B" "A"
LRANGE learn:recent 0 -1
LPUSH learn:recent "C"
LRANGE learn:recent 0 2
LREM learn:recent 0 "A"
LTRIM learn:recent 0 9
LPOP learn:recent
```

After RPUSH the sequence is `[A, B, A]`; after LPUSH C it is `[C, A, B, A]`. LRANGE 0 2 returns the first three. LREM with count 0 removes ALL occurrences of A; LTRIM keeps only indices 0 through 9. LPOP removes the first element; RPOP removes the last. LLEN gives the length.

Recently viewed pattern: remove the existing ID → push it to the front → keep the first ten → refresh expiry. This produces a bounded, recency-ordered list without duplicate products. The complete sequence needs atomic execution when concurrent requests can update it.

Queue idea: RPUSH plus LPOP gives first-in-first-out order. A raw list alone is not a complete reliable job system; Celery adds worker, delivery and retry behavior. Use lists here to understand ordering and ends.

Sets: uniqueness matters, order does not

A set contains distinct members. Adding an existing member does not create another copy. It has no position-based order, so it is unsuitable for “most recent first.”

```
SADD learn:wishlist:user:7 "A" "B" "A"
SMEMBERS learn:wishlist:user:7
SISMEMBER learn:wishlist:user:7 "A"
SCARD learn:wishlist:user:7
SREM learn:wishlist:user:7 "B"
```

The set contains A and B once each. SMEMBERS returns all members in unspecified order. SISMEMBER checks membership; SCARD counts distinct members; SREM removes members. SUNION combines sets; SINTER finds common members; SDIFF finds members in the first set but not the others.

Need	Choose	Why
Recent items in viewing order	List	Explicit sequence and efficient operations at the ends
Distinct selected product IDs	Set	Uniqueness and membership checks
Highest-scored products	Sorted set	Order is derived from numeric scores

Read size matters: LRANGE 0 -1 and SMEMBERS retrieve everything. Bound results when collections can be large. Most missing collection keys behave like empty collections for these reads; this differs from a missing GET returning nil.

06 · Sorted sets and choosing a type

A score determines the order

A sorted set maps each unique member to a numeric score. The member can be a product UUID; the score can be views or a timestamp. Re-adding the same member updates its score rather than creating a duplicate member.

```
ZADD learn:popular 10 "A" 30 "B" 20 "C"
ZRANGE learn:popular 0 -1 WITHSCORES
ZINCRBY learn:popular 15 "A"
ZRANGE learn:popular 0 1 REV WITHSCORES
ZSCORE learn:popular "A"
ZCARD learn:popular
ZREM learn:popular "C"
```

Initially the ascending order is A(10), C(20), B(30). After ZINCRBY, A has 25. The highest two are B(30), A(25). REV gives descending order; 0 1 means the first TWO positions. ZSCORE reads one member's score; ZCARD counts members; ZREM removes them. Equal scores use member lexicographical order as a tie-breaker; do not assume insertion order.

YOUR QUESTION — "What does ZRANGE return?" Members by default. In Python, `r.zrange(key, 0, 9, desc=True)` gives up to ten members, usually bytes with your client. Add `withscores=True` for `(member, score)` pairs. A missing sorted set gives `[]`.

```
r.zrange("learn:popular", 0, 1, desc=True, withscores=True)
# With default decoding: [(b"B", 30.0), (b"A", 25.0)]
```

Why this fits rankings and sliding logs

For popularity, increment a product member's score. For a sliding-window request log, each request needs a distinct member ID and its timestamp as score; equal timestamps must not collapse two requests into one member. Score-range operations can remove old timestamps.

Your daily ranking stores a date in the KEY, not in each score. The score is today's request count. A different day's key produces a separate ranking. Sorted sets, ZRANGE.

Choose by the operation you need

Data/access pattern	Suitable type	Core operation
Prepared catalog response	String / Django cache value	Read or replace the whole result
Single counter	String	INCR
Object with independently updated fields	Hash	HGET / HSET / HINCRBY
Ordered recent IDs	List	LPUSH / LREM / LTRIM
Unique membership	Set	SADD / SISMEMBER
Ranking or timestamp-ordered members	Sorted set	ZINCRBY / ZRANGE

Wrong type: GET on a hash or HGET on a string raises WRONGTYPE. Use TYPE to inspect. When changing a feature's stored type, choose a new versioned key or deliberately migrate old data; changing only the Python code does not transform stored values.

07 · Python clients and connection pools

Connect once, reuse the client

```
import redis

r = redis.Redis.from_url(
    "redis://127.0.0.1:6379/0",
    socket_connect_timeout=1,
    socket_timeout=1,
    max_connections=20,
)
r.set("learn:name", "Mouse", ex=60)
name = r.get("learn:name")
```

In the URL, 127.0.0.1 is the host, 6379 the port and /0 the logical database. The default reply is bytes, such as `b'Mouse'`; `name.decode('utf-8')` produces a string. With `decode_responses=True`, the client decodes text replies automatically. Do not apply `.decode()` again in that case.

The pooling terms you asked about

Connection: a reusable communication channel between Python and Redis. **TCP:** the networking protocol carrying an ordered byte stream. **Socket:** an OS endpoint for sending/receiving. **Pool:** manages connections. **Idle:** available. **In use:** borrowed by an operation. **Concurrent operations:** operations whose execution overlaps.

For a normal command: borrow a connection → send command → receive reply → return connection. Reuse avoids establishing a fresh connection for every GET/SET. Connections are normally created as needed, not all at import time.

`max_connections=20` caps THIS pool; it does not restrict the site to 20 users. If all are borrowed, the ordinary pool raises on exhaustion; a blocking pool can wait. `socket_connect_timeout` bounds establishing a connection; `socket_timeout` bounds socket I/O waits, not the full HTTP request.

r versus cache

> **YOUR QUESTION — "What is the difference between r and cache?"** `r` exposes native Redis structures and Lua. Django's `cache` exposes a backend-independent cache API and handles key prefix/version plus Python-value serialization. With RedisCache configured, both reach Redis, but their keys and stored encodings can differ.

```
from django.core.cache import cache
cache.set("product:101", {"name": "Mouse"}, timeout=60)
product = cache.get("product:101")
```

A direct `r.set` cannot store that dictionary without serialization, for example JSON. Django's cache format should not be assumed to be JSON. Direct EX is `ex=60`; Django uses `timeout=60`.

Your central client: throttles, history, ranking and locks import `config.redis_client`. This shares one direct pool per application process. Django cache uses its own pool; separate web/worker processes have separate pools. Use `cache.make_key(...)` for direct feature keys needing Django's prefix; pass logical keys to cache.get/set so the prefix is not applied twice.

08 · Caching: the idea and the key rules

Problem → solution

Repeated requests load the same product/delivery data and serialize it again. A cache stores a reusable result for a limited time. **Cache-aside** means the application checks the cache and loads the source database only when needed.

```
Request → cache.get(key)
            ├─ hit  → return cached data
            └─ miss → database → serialize → cache.set(TTL) → return
```

Cold cache: the needed entry is absent. **Warm cache:** it is present. Those describe how a request is served, not different HTTP methods. Setting CACHES only chooses storage; your view must implement the read/hit/miss flow. A cache hit can still require authentication/session work.

YOUR QUESTION — "Does an empty list count as None?" No. `[]` can be a valid cached result. Both are falsy, so `if data:` mistakenly treats an empty cached result as a miss. Use `data is not None` when None is your miss marker; use a sentinel if caching None itself is required.

YOUR QUESTION — "If reading cache fails, don't we write?" Your pattern skips the response-cache write for that request. A miss means Redis answered with no value; a failure means an exception. Skipping a likely failing write avoids extra delay. The next request starts by trying Redis again.

Build keys for the complete representation

An expanded delivery response and a plain response need different keys. So do different users when caching private data. Do not put every user under one public key. Search/filter/sort/page inputs also change responses; your project currently caches the default product list and bypasses search caching.

Expiry and invalidation

TTL bounds the lifetime of a stale copy; it does not know that PostgreSQL changed. **Invalidation** deletes affected keys after a successful write. Product edits/deletes invalidate list and detail caches; checkout invalidates stock-bearing product responses. Imports/admin/direct writes need their own write-path handling.

Register invalidation with `transaction.on_commit(...)` so rollback does not publish a successful cache state. Registering does not commit; remaining database code runs first. Outside a transaction under normal autocommit, the callback runs immediately. This is a database timing rule, not a background task mechanism.

Your caching values: list and delivery responses use 300 seconds; product details use 300–360 seconds of jitter. Checkout continues validating authoritative prices/stock from PostgreSQL.

Read the next page's code flow

Read cache → return hit → obtain DRF database response on miss/failure → attempt cache write only when available → return the existing Response. Store `response.data`, not the Response object. The snippet uses a public endpoint with an expansion-specific key and network-failure fallback.

09 · Complete DRF caching snippet

This mirrors your delivery-options view. It assumes Django RedisCache is configured in CACHES and these model/serializer modules already exist.

```
import logging
from django.core.cache import cache
from redis.exceptions import ConnectionError as RedisConnectionError
from redis.exceptions import TimeoutError as RedisTimeoutError
from rest_framework import generics
from rest_framework.response import Response
from .models import DeliveryOption
from .serializers import DeliveryOptionSerializer

logger = logging.getLogger(__name__)

class DeliveryOptionListView(generics.ListAPIView):
    queryset = DeliveryOption.objects.all()
    serializer_class = DeliveryOptionSerializer

    def list(self, request, *args, **kwargs):
        expanded = (
            request.query_params.get("expand") == "estimatedDeliveryTime"
        )
        key = f"delivery-options:v1:expanded:{int(expanded)}"
        cache_available = True

        try:
            data = cache.get(key)
        except (RedisConnectionError, RedisTimeoutError):
            logger.warning("Delivery cache read failed")
            data = None
            cache_available = False

        if data is not None:
            return Response(data)

        response = super().list(request, *args, **kwargs)

        if cache_available and response.status_code == 200:
            try:
                cache.set(key, response.data, timeout=300)
            except (RedisConnectionError, RedisTimeoutError):
                logger.warning("Delivery cache write failed")

        return response
```

Hit: Response(data) creates a response with default HTTP 200. **Miss:** super().list queries/serializes and already returns a Response; return it directly. Wrapping it in Response(response) would wrap a Response object instead of raw serializable data.

Warnings appear where logging handlers send them—normally the running backend terminal here. The snippet catches Redis connection/timeout failures; other error types are not covered by those handlers. Cache failure does not prevent the database response.

10 · Fixed and sliding windows

Fixed window: count within one period

Idea: permit at most five requests in each 60-second window. Store one counter per identity/window. **Problem solved:** a cheap shared limit on request volume, with very little state.

Flow: identify client → select current window → atomically increment its counter and ensure expiry → allow counts up to five → otherwise return 429 and the remaining time until reset. Keep increment/expiry coordination together, using the atomicity rules on the next page.

Two window choices: clock-aligned windows reset at boundaries such as 12:01:00; the key can include the minute's identifier. A first-request-anchored window starts when the initial counter is created and expires 60 seconds later. In the latter design, refreshing expiry on every request would turn it into a different policy and could keep delaying reset.

Boundary problem: five requests at 12:00:59 and another five at 12:01:01 pass a clock-aligned five-per-minute limit. Each minute obeys the rule, but ten requests arrive within about two seconds. Fixed windows constrain each chosen period, not every possible rolling 60-second interval.

Sliding-window log: count the recent interval

Idea: allow at most five accepted requests in the preceding 60 seconds, evaluated anew on every request. Store accepted requests in a sorted set: a unique request ID is the member, and its timestamp is the score.

Atomic code flow: get current time → remove timestamps at or before now minus 60 seconds → count remaining members → if fewer than five, add this request and allow → otherwise reject and calculate when the oldest retained request leaves → set expiry for inactive-key cleanup.

Example: at 12:00:50 the five retained requests include an oldest request at 12:00:10. Another is rejected. That oldest request leaves at 12:01:10, so the next slot is about 20 seconds away. Slots return one by one as timestamps age out; a clock-minute boundary does not clear the whole log.

> **YOUR QUESTION — “Should wait be a duration minus elapsed time?”** For the sliding log, yes: wait = window − (now − oldest retained request time), rounded up to seconds. Token buckets use the missing-token balance instead. The estimate does not reserve the next slot against other clients sharing that identity.

Practical choice

Fixed: simplest and constant-sized state; permits boundary bursts. **Sliding log:** exact rolling count; stores individual accepted requests and performs cleanup. **Token bucket:** compact balance/timestamp state; explicitly allows bursts with gradual refill. A sliding-window counter uses weighted window counts as an approximation; it is different from the exact log described here.

Use Redis time and atomic read/decide/write logic for shared state. Unique log members prevent equal timestamps collapsing multiple requests into one. When switching the stored type, version the key to avoid WRONGTYPE. Redis counter/rate-limiter patterns.

11 · Rate limiting and token buckets

Basic idea and problem solved

Rate limiting controls how much work one client can request. Your product-search limiter protects uncached database searches. It identifies signed-in users by user ID and guests by IP, then returns HTTP 429 when allowance is exhausted. It is separate from authentication and checkout idempotency.

Approach	How allowance is represented
Fixed window	Counter for one time period; resets at the boundary
Sliding-window log	Recent request timestamps; old requests leave the window
Token bucket	Stored balance replenishes with time, up to a capacity

How your token bucket solves it

Each request costs one token. Capacity permits a short burst; refill rate controls sustained traffic. Your user policy is capacity 5 and refill 5/60 tokens per second; guests get capacity 3 and refill 3/60. This is gradual refill, not a strict rolling-minute request count.

Code flow: receive search → choose identity/policy → Lua reads Redis time and hash state → calculate refill up to capacity → consume one token if available, otherwise calculate wait → save balance/timestamp and refresh expiry → return allowed/wait → DRF allows or emits 429 with Retry-After.

YOUR QUESTION — “Why only search?” It was the focused learning feature and search bypasses the response cache. Other routes can have separate policies. IP-based guests can share an allowance behind NAT; forwarded IP headers should only be trusted from a configured proxy.

YOUR COUNTERQUESTION — “Why not INCR then immediately EXPIRE in Python? Can't Lua fail too?” Separate commands have a crash/interleaving gap. Lua keeps the read/decide/write flow together, but a later script error does not roll back earlier writes. Validate inputs before writing; keep scripts short.

Wait and expiry are different

Refilled balance = min(capacity, old balance + elapsed × refill rate). If below one token, wait = missing tokens ÷ refill rate, rounded up for HTTP seconds. No background process spawns tokens; the next request computes what accumulated.

YOUR QUESTION — “Isn't wait token period minus elapsed?” Only when the old balance was zero. At 1 token/12 seconds, an old 0.25 plus six seconds' refill gives 0.75; missing 0.25 needs **3 seconds**, not 12 − 6 = 6 seconds. A sliding log instead waits for the oldest request to leave its window.

YOUR QUESTION — “How is TTL chosen?” Capacity ÷ refill rate, rounded up: 60 seconds here. After that much inactivity, an empty bucket would be full, so forgetting its state is safe. Each request refreshes expiry. TTL cleans up inactive identities; it does not drive refill.

Your search limiter fails open on connection/timeout errors. That preserves browsing availability; stricter endpoints can choose another policy. Native INCR is atomic, batching reduces network trips, and Lua provides atomic conditional flow; none implies SQL-style rollback.

12 · Sessions, recent history and popularity

Cached sessions

Problem: multiple web processes need shared login state with efficient reads. **Solution:** Django cached_db stores sessions in PostgreSQL and caches them in Redis. **Flow:** browser supplies session cookie → Django loads session from cache or database → authenticated request runs. Evicted/missing data can be rebuilt from PostgreSQL; a network exception is a separate failure path.

> **YOUR QUESTION — "Do tests also use Redis cached sessions?"** Your final `config.test` does: cached_db sessions, temporary PostgreSQL database and Redis keys with a `:test` prefix. Earlier DB-only/in-memory examples were replaced. Cache prefixes do not isolate Celery queues.

Recently viewed products

Problem: display a user's last ten distinct products in recency order. **Solution:** a bounded Redis list per signed-in user. **Write flow:** remove existing ID → push to front → trim to ten → refresh 30-day expiry, all in one Lua script. **Read flow:** list IDs → load current database products → restore Redis ID order → serialize.

> **YOUR QUESTION — "What if the product is already cached?"** Record the view before returning a successful cache hit AND after a successful database lookup. A helper keeps the same authentication and optional-write error handling on both paths.

> **YOUR QUESTION — "Why decode IDs and rebuild ordered_products?"** Default Redis replies are bytes; dictionary keys built from database IDs are strings. Decode and normalize first. `filter(pk__in=ids)` does not preserve ID order, so reconstruct it using an ID-to-product map and skip deleted products.

> **YOUR QUESTION — "If the helper has a Redis error, what returns and where do we handle it?"** An exception propagates rather than returning None. Your view chooses the fallback: the history endpoint returns an empty list on connection/timeout failure; detail tracking logs and lets the product response continue. Handle/log once at that policy boundary.

Daily most-viewed products

Problem: rank products by today's detail requests. **Solution:** one sorted set for each UTC date, product ID as member and request count as score. **Flow:** successful detail view, including cache hit → ZINCRBY → refresh expiry → top-ten IDs → hydrate database objects in score order.

> **YOUR QUESTION — "Why TTL times two?"** Two days is retention after the key's last write. The date in the key defines the day; it is not a rolling two-day ranking. Guests and repeat requests count, so this measures requests, not unique visitors.

All three features have distinct failure consequences: cached session data has a database record; recent/popularity tracking can be skipped; a product response should not fail merely because optional tracking could not be written.

13 · Checkout idempotency

Basic idea

Idempotency makes a retry of the same intended operation return its existing outcome. Here the operation is a checkout identified by a client-supplied Idempotency-Key. A retry reuses the key; a different purchase gets a new key.

Problem solved

A customer submits checkout; the server commits an order, but the response is lost. Without retry recognition, submitting again could create another order and deduct stock twice. A rate limit cannot tell whether two allowed requests represent the same purchase.

How your design solves it

PostgreSQL stores a unique `(user, key)` mapping to the order. Redis caches completed results for faster lookup, but the database mapping preserves recognition after Redis expiry/outage. The key is scoped to the user, not globally shared across customers.

Working code flow

1. Validate the nonempty header and its 128-character maximum; form the response-cache key using user, request key and expansion choice.

2. A cached completed result returns immediately with 201.

3. Otherwise start a database transaction and lock the user. Find/create the unique CheckoutRequest.

4. If it already links to an order, serialize and return that order. Do not deduct inventory or empty the current cart again.

5. For a new request, lock cart/products, validate authoritative stock, deduct inventory, create order/items, link the request and clear the checked-out cart rows.

6. Commit all database changes together; then invalidate product caches, enqueue confirmation and cache the result when Redis is available.

If the transaction fails, its new request mapping and order/stock changes roll back together. If the response is lost after commit, a retry finds the mapping. An existing mapping without a linked order returns 409 in your code.

> **YOUR QUESTION — “Won't saving every order again waste space?”** The order already exists. The additional record stores references, request key and timestamp—not another full order copy. You chose to retain it. Deleting the mapping after a retention window also ends recognition of that old key.

Keep these mechanisms distinct

Idempotency: recognizes repeated operations. **PostgreSQL row locks:** serialize conflicting database writes and stock checks. **Redis cache locks:** coordinate rebuilding a shared cached result. These mechanisms solve different problems and can work together.

Your result-cache lifetime is one hour; the mapping survives longer in PostgreSQL. Reusing a key for a changed cart still means the original checkout. The order can be serialized with different expansion options, so replay is not necessarily the identical bytes of the original HTTP response. Frontend retries must preserve the key for the same purchase.

14 · Cache locks and TTL jitter

Basic idea and problem solved

A popular cached list expires just before many requests arrive. All see a miss, query PostgreSQL and serialize the same result: a cache stampede. A Redis lock lets one current owner rebuild while other requests briefly wait for the filled cache.

How the lock works

SET with NX creates the lock only if absent; EX gives it a lease. The stored random token identifies the owner. A process owns the lock only after SET succeeds. If it crashes, expiry eventually frees the lock. Release compares the token and deletes the lock atomically so an expired owner cannot delete someone else's new lock.

Working code flow

1. Read product-list cache; return on hit.
2. On miss, try acquiring the lock with a token and ten-second expiry.
3. If acquired, recheck cache: another worker may already have filled it.
4. Still missing: read/serialize the database result, cache it, return it.
5. In finally, release only if the stored token still matches.
6. If another owner holds the lock, poll cache briefly. Your roughly one-second wait budget then falls back to PostgreSQL, preserving availability.

This reduces duplicate rebuilding; expiry and the wait fallback still permit extra database queries.

> **YOUR QUESTION — “What does releasing mean?”** Deleting the separate LOCK key after checking ownership. It does not delete the cached product data.

> **YOUR COUNTERQUESTION — “A expires, B acquires, then A finishes?”** A's token differs from B's, so A cannot delete B's lock. A can still finish and write its result: safe release protects deletion, not stale-owner writes. The lease does not stop Python when it expires.

> **YOUR QUESTION — “Does finally run even after return?”** In normal execution, Python evaluates the return value, runs finally, then returns to the caller. Uncaught cleanup errors can replace that return; abrupt process death can skip cleanup. The expiry handles abandoned locks.

Jitter: spread many keys' expirations

Problem: many product-detail keys are populated together and later expire together.
Solution: add a small random amount to their TTL. Your detail TTL is 300–360 seconds, so expiry is spread over a minute.

> **YOUR QUESTION — “Jitter on list or individual products?”** Individual products first, because many keys can expire together. A single hot list still needs coordination when it expires; its rebuild lock addresses that burst.

Remaining limit: reader loads old data → writer commits and invalidates → reader writes its old result. Post-commit invalidation and your reader lock do not prevent this race. A generation token in keys can direct new readers to a newer generation; that is a separate improvement.

15 · Celery from first principles

Basic idea and problem solved

Sending email during checkout makes the HTTP request wait for a potentially slow external operation. Celery moves that work to a separate worker process. Redis acts as the broker holding task messages until a worker receives them.

```
Django producer → Redis broker → Celery worker → email backend
Celery Beat     → Redis broker → Celery worker → periodic recovery
```

Task: registered Python function. **Message:** task name and serialized arguments. **Producer:** code publishing it. **Broker:** message transport/queue. **Worker:** process importing and executing task code. **Beat:** scheduler publishing periodic tasks. A result backend, when configured, stores task results; it is separate from the broker's role. Your email tasks ignore Celery results and track business state in PostgreSQL.

Working code flow

1. Checkout saves the order in a database transaction.
2. on_commit registers publication after successful commit, passing the order ID.
3. The callback queues the task; the HTTP request need not wait for sending.
4. The worker receives the name/arguments, loads the committed order and runs the task.
5. The task records its outcome; later failures can be retried or recovered.

> **YOUR QUESTION — "What if we enqueue immediately without delay?"** Calling `task(order_id)` runs synchronously in the caller. `.delay(order_id)` queues immediately; its name does not mean sleeping. Publishing before commit lets the worker query too early or run work for a rolled-back order. on_commit controls timing; Celery controls background execution.

Run the processes

```
cd ~/projects/ecommerce_project/ecommerce_backend
source .venv/bin/activate
celery -A config worker --loglevel=INFO
# Separate terminal: same directory and environment
celery -A config beat --loglevel=INFO
```

Worker and Beat are separate processes. Beat cannot execute the task without a worker. Use one scheduler for the same schedule. Restart workers after changing tasks/config so they load the new code.

Doubts from setup that matter to Celery

Settings not configured: use the exact DJANGO_SETTINGS_MODULE spelling; your earlier typo omitted the final S in SETTINGS. **Unregistered task/KeyError:** the running worker must import that task name; check autodiscovery and restart after adding/renaming tasks.

Development/test/load workers use the Django settings they start with. Cache-key prefixes do not route messages to matching databases; isolate broker namespaces/queues and workers deliberately. Redis /0 and /1 on one server still share memory and eviction policy. A committed order followed by failed publishing needs recovery; on_commit alone cannot guarantee publication.

16 · Email retries and recovery: the flow

What the email task is doing

Problem: sending can fail temporarily, checkout should not be undone by email failure, and duplicate task execution should not casually send again. **Solution:** enqueue after commit, track sending state on Order, serialize attempts with a row lock, retry selected failures and periodically recover unfinished work.

Working code flow

1. Worker loads and locks the Order inside atomic.
2. Skip if already sent, user has no email, or the saved attempt limit is reached.
3. Increment attempts and save last-attempt time; try sending.
4. If one message succeeds, record the sent timestamp.
5. If sending raises, temporarily keep that exception, commit the attempt record, then raise it outside the transaction. Celery retries only the configured exception types.
6. Beat periodically finds eligible unsent orders and queues another attempt when needed.

> **YOUR QUESTION — “Why catch exc now when we didn't earlier?”** An exception escaping atomic immediately would undo the attempt increment. Saving the exception until after commit preserves the failed attempt. It does not turn every error into an automatic retry; only configured transient errors qualify.

Retry and recovery are two mechanisms

Automatic retry: selected connection/disconnection/timeout failures schedule retries after 5, 10 and 20 seconds. Three retries means up to four runs including the first. **Recovery:** every minute, find unsent orders with email, saved attempts below five and a last attempt at least five minutes old—or no attempt yet. A saved guard caps normal sending at five attempts across task chains.

> **YOUR QUESTION — “Why last_attempt <= cutoff?”** At 12:10 the five-minute cutoff is 12:05. A 12:03 attempt is old enough; 12:08 is recent. Older timestamps are smaller. The cooldown gives recovery a different role from short automatic retries.

> **YOUR QUESTION — “Does this send real mail? Do timeouts need SMTP?”** Your console backend prints in the worker terminal and can mark success without inbox delivery. SMTP timeouts matter when an SMTP connection exists; a local development server is sufficient. Real delivery needs a configured SMTP/provider backend. In your Django 6.1 setup, use MAILERS consistently rather than mixing it with deprecated EMAIL_BACKEND.

What the design does and does not guarantee

The row lock makes duplicate workers check updated state in sequence. But sending to an external system and saving success are not atomic: mail may succeed, then the database transaction/process can fail before recording it, allowing a duplicate later. The lock is held during sending, so an SMTP timeout bounds a slow network wait. A periodic recovery scan can repair missing publication for this email use case; it is not exactly-once delivery.

Revision sources: saved learning conversation and the current ecommerce modules. Key references: Django transactions, Django email, Celery retries.

17 · Memory limits and eviction

Limit the memory budget

`maxmemory` is the configured memory threshold Redis uses for eviction decisions. A value such as 256mb is a deployment choice, not a recommendation for this project. **0 means no configured Redis limit**, not unlimited machine memory. Allow headroom for process and persistence/replication overhead; it is not a hard cap on every byte of process RAM.

Expiration removes a key because its TTL elapsed. **Eviction** removes a key early to make room under memory pressure. **maxmemory-policy** determines eligible keys and the selection strategy; LRU/LFU are approximate policies.

Policy	Behavior under memory pressure
noeviction	Reject memory-growing writes rather than evict keys; reads can continue
allkeys-lru	Evict from all keys, favoring least recently used
allkeys-lfu	Evict from all keys, favoring least frequently used
volatile-lru	Evict only keys with expiry, favoring least recently used
volatile-ttl	Evict only expiring keys, favoring the shortest remaining TTL

With a volatile policy, no eligible expiring keys can mean writes fail instead. **LRU asks “used how recently?”; LFU asks “used how often?”** An eviction candidate can have a valid TTL and still disappear.

Apply the policy to the workload

A product cache can be rebuilt. Losing a broker message, active lock or rate-limit counter has different consequences. Your Redis /0 and /1 share memory and policy, so logical database numbers do not protect Celery queues from eviction. Separate instances allow different policies; noeviction still requires capacity monitoring and publish-error handling. An OOM write error is not a connection/timeout exception.

```
CONFIG GET maxmemory
CONFIG GET maxmemory-policy
INFO memory
INFO stats
```

These inspect rather than change settings. Watch used_memory, the configured maxmemory and changes in evicted_keys. A cache miss after eviction should rebuild from PostgreSQL. Redis eviction policies.